



Frontend Integration Guide

Complete backend integration reference for all 8 pages
of the SASTRA web application.

[sastra-landing.html](#) [signin.html](#) [signup.html](#) [dashboard.html](#) [new-scan.html](#) [scan-results.html](#)
[scan-history.html](#) [settings.html](#)

Table of Contents

1. Project Overview & Architecture
2. sastra-landing.html — Landing Page
3. signin.html — Sign In Page
4. signup.html — Sign Up Page
5. dashboard.html — Security Dashboard
6. new-scan.html — New Scan Page
7. scan-results.html — Scan Results Page
8. scan-history.html — Scan History Page
9. settings.html — Settings Page
10. Global Auth & sessionStorage Reference
11. Complete API Endpoint Summary
12. Database Schema Reference

1. Project Overview & Architecture

SASTRA is a web-based Static Application Security Testing (SAST) framework designed to analyse Java, Python, and C++ source code for security vulnerabilities. The system uses rule-based regex and AST parsing as the primary detection method, with ML integration for severity prediction and false-positive filtering, and an external LLM API for AI-powered code fix suggestions. Results are stored in a SQLite database and users can export reports as PDF files.

Technology Stack

Frontend	8 static HTML files with CSS and vanilla JavaScript. All pages share the same gradient theme, glassmorphism cards, Sora + DM Sans fonts, and sticky frosted navbar.
Backend	Python (Flask or FastAPI recommended). Rule-based static analysis engine + ML classifier + LLM API calls.
Database	SQLite — stores users, scans, and vulnerability findings.
Auth	Username + hashed password (bcrypt). JWT or session tokens for authenticated routes.
LLM	External LLM API (e.g. OpenAI, Anthropic) for code fix suggestions.
PDF Export	Server-side PDF generation (reportlab recommended) returning a binary stream.

Page Navigation Flow

Page	File	Access	Links To
Landing	sastra-landing.html	Public	signin.html, signup.html
Sign In	signin.html	Public	dashboard.html (on success), signup.html
Sign Up	signup.html	Public	dashboard.html (on success), signin.html
Dashboard	dashboard.html	Auth required	new-scan.html, scan-history.html, settings.html
New Scan	new-scan.html	Auth required	scan-results.html (on success)
Scan Results	scan-results.html	Auth required	new-scan.html, dashboard.html
Scan History	scan-history.html	Auth required	new-scan.html, scan-results (drawer)
Settings	settings.html	Auth required	signin.html (logout / delete)

2. Landing Page

sastra-landing.html

The public-facing entry point. No authentication required. Introduces SASTRA to new visitors and directs them to sign in or register. No API calls are made from this page.

Page Sections

Navbar

Fixed top bar with SASTRA shield logo (links home), Login button (→ signin.html), Sign Up button (→ signup.html).

Hero Section

Large headline, two-line subtitle describing the product, two CTA buttons, and a small footnote. Decorative animated gradient blobs in the background.

Badge

Animated "AI-Powered Security Analysis" pill with pulsing green dot — purely decorative.

Interactive Elements & Outputs

Element	Type	Action / Output
Login button	Button	Navigates to signin.html. No API call.
Sign Up button	Button	Navigates to signup.html. No API call.
Start Scanning  button	Button	Navigates to signup.html. No API call.
Learn More button	Button	Placeholder — no current target. Wire to a features/about section or page.
SASTRA logo	Link	Reloads sastra-landing.html.

Integration Notes

- No sessionStorage keys are read or written on this page.
- This page has no protected routes — it should always be accessible without a token.
- The "Learn More" button has no current destination. You may link it to a features section or leave as a scroll anchor.

3. Sign In Page

signin.html

Handles user authentication. Collects username and password, calls the login API, stores the returned token and username in sessionStorage, then redirects to the dashboard.

Page Sections

Navbar

SASTRA logo only (links to landing page). No nav links shown on auth pages.

Sign In Card

Centred glassmorphism card containing the form heading "Sign In", subtitle "Welcome back", two input fields, the Login button, and a "Don't have an account? Sign up" link.

Interactive Elements & Outputs

Element	Type	Action / Output
Username input	Input (text)	id="username". Captures the username string. Red box-shadow applied if empty on submit.
Password input	Input (password)	id="password". Captures the password string. Red box-shadow applied if empty on submit.
Login button	Button	Calls handleLogin(). Validates non-empty fields. On success: stores sastra_user and sastra_token in sessionStorage.
Sign up link	Anchor	Navigates to signup.html.

Required Backend API Calls

POST

/api/auth/login

Authenticates the user. Called inside handleLogin() after client-side validation.

Request Body:

```
{ "username": "string", "password": "string" }
```

Response:

```
200: { "token": "jwt_string", "username": "string" } 401: { "error": "Invalid credentials" }
```

Integration Notes

- Store the JWT in sessionStorage under key "sastra_token". Include it as Authorization: Bearer {token} in all subsequent API calls.
- The frontend does NOT currently call a logout endpoint — add sessionStorage.clear() and a POST /api/auth/logout call to the Log Out button in the navbar.
- Passwords must be hashed server-side using bcrypt before storing in SQLite (FR-16).

4. Sign Up Page

signup.html

Handles new user registration. Validates that passwords match client-side, then calls the register API. On success stores the session token and redirects to the dashboard.

Page Sections

Navbar

SASTRA logo only (links to landing page).

Sign Up Card

Centred card containing "Sign Up" heading, "Create an Account" subtitle, three input fields, Create Account button, and "Already have an account? Sign in" link.

Interactive Elements & Outputs

Element	Type	Action / Output
Username input	Input (text)	id="username". Required. Red highlight if empty.
Password input	Input (password)	id="password". Required. Red highlight if empty.
Confirm Password input	Input (password)	id="confirm-password". Must match password. Red highlight if mismatch.
Create Account button	Button	Calls handleSignup(). Client-side validation: non-empty username, non-empty password, password match.
Sign in link	Anchor	Navigates to signin.html.

Required Backend API Calls

POST

/api/auth/register

Creates a new user account. Password must be hashed server-side before storage.

Request Body:

```
{ "username": "string", "password": "string" }
```

Response:

```
201: { "token": "jwt_string", "username": "string" } 409: { "error": "Username already exists" }
```

Integration Notes

- The frontend validates password match before making the API call — the backend should also validate and return a 400 if they differ.
- Usernames must be unique. Return HTTP 409 Conflict if the username is already taken.
- After successful registration, the user should be immediately authenticated (token returned) and land on the dashboard without needing to sign in again.

5. Security Dashboard

dashboard.html

The main authenticated landing page. Shows a real-time security summary with 4 stat cards and a Recent Scans table. All data is currently hardcoded and must be replaced with live API calls.

Page Sections

Sticky Navbar

Frosted glass bar with logo, four nav links (Dashboard active), welcome greeting (reads sastra_user from sessionStorage), username pill button, and dark Log Out button.

Dashboard Header

"Security Dashboard" title + subtitle. "Start New Scan" button top-right.

4 Stat Cards

Glassmorphism cards: (1) Total Scans with +12% trend, (2) Total Vulnerabilities across all scans, (3) Critical + High Issues with warning label, (4) Last Scan showing filename and date.

Recent Scans Section

Table showing the 4 most recent scans. Each row: file icon, filename, status badge, language, file size, date, issue count, severity tags (Critical/High/Medium). "View All →" link goes to scan-history.html.

Interactive Elements & Outputs

Element	Type	Action / Output
Dashboard nav link	Nav link	Active state. Reloads dashboard.html.
New Scan nav link	Nav link	Navigates to new-scan.html.
Scan History nav link	Nav link	Navigates to scan-history.html.
Settings nav link	Nav link	Navigates to settings.html.
Log Out button	Button	Redirects to signin.html. Add: sessionStorage.clear() + POST /api/auth/logout.
Start New Scan button	Button	Navigates to new-scan.html.
View All → link	Anchor	Navigates to scan-history.html.
Scan row (each)	Clickable row	Currently no action. Wire to scan-results.html?id={scan_id}.

Required Backend API Calls

GET

/api/scans/stats

Returns aggregate statistics for the 4 stat cards. Called on page load.

Response:

```
{ "total_scans": 5, "total_vulnerabilities": 45, "critical_high_count": 22, "last_scan": { "filename": "auth-service.py", "scanned_at": "2 days ago" } }
```

GET

/api/scans/recent?limit=4

Returns the 4 most recent scans for the Recent Scans table.

Response:

```
{ "scans": [ { "id": "uuid", "filename": "auth-service.py", "language": "Python", "file_size": "45 KB",  
"status": "completed", "scanned_at": "2 days ago", "critical": 2, "high": 4, "medium": 4, "issues_total": 12  
} ] }
```

Integration Notes

- All 4 stat card values are hardcoded in the HTML — replace the text nodes with values from the `/api/scans/stats` response.
- The Recent Scans table rows are hardcoded — dynamically generate rows from the `/api/scans/recent` response.
- The username greeting reads `sessionStorage.getItem("sastra_user")` — this is set during login/signup.
- The Log Out button should call `POST /api/auth/logout` to invalidate the server-side session, then clear `sessionStorage` and redirect.

6. New Scan Page

new-scan.html

The core scan submission page. Supports two input methods: file upload (single file or ZIP) and direct code paste. On submission, the frontend shows an animated progress overlay, then redirects to the scan results page.

Page Sections

Navbar

New Scan link is active. Same sticky frosted nav as all authenticated pages.

Page Header

"New Scan" title + subtitle. No action buttons.

Method Toggle

Two pill buttons: "Upload File" (default active) and "Paste Code". Toggling shows/hides the relevant input area.

Upload File Area

Full drag-and-drop zone. Accepts .py, .java, .cpp, .c, .zip, .tar.gz. Click opens native file picker. Shows filename + size on selection. "Remove file" clears selection.

Paste Code Area

Hidden until "Paste Code" is selected. Language selector (Python / Java / C++). Large textarea for pasting source code.

Run Scan button

Full-width dark button. Validates that file is selected or code is non-empty before submitting. Shows animated scanning overlay with step-by-step progress labels.

Scanning Overlay

Full-screen modal with dual spinning rings, animated progress bar 0-100%, live step labels (Parsing, Static analysis, CVE check, AI fix generation, etc.), then redirects to scan-results.html.

Interactive Elements & Outputs

Element	Type	Action / Output
Upload File toggle	Button	Shows drop zone, hides paste area. Sets method="file".
Paste Code toggle	Button	Shows paste area, hides drop zone. Sets method="paste".
Drop zone / file picker	Input (file)	id="file-input". Accepts .py/.java/.cpp/.c/.zip/.tar.gz. On select: shows filename (id="chosen-na
Remove file button	Button	Clears file input and resets drop zone to default state.
Python / Java / C++ buttons	Toggle buttons	Sets the selected language for paste mode. Value read at submit time via .querySelector(".lar
Code textarea	Textarea	id="code-input". Stores pasted source code. Read at submit.
Run Scan button	Button	Validates input → saves scan metadata to sessionStorage("sastra_scan") → POSTs to /api/s

Required Backend API Calls

POST	/api/scans/run
------	----------------

Submits the code for analysis. Send as multipart/form-data. The backend runs preprocessing, rule-based analysis, ML severity prediction, and LLM fix generation.

Request Body:

```
multipart/form-data: file: (binary) – for file/ZIP upload OR code: (string) – for paste mode language:
"Python" | "Java" | "C++"
```

Response:

```
200: { "scan_id": "uuid", "filename": "auth-service.py", "language": "Python", "file_size": "45 KB",
"status": "completed" | "failed" } 400: { "error": "Unsupported file type" | "File too large" }
```

Integration Notes

- The frontend stores scan metadata in sessionStorage("sastra_scan") as JSON: { filename, lang, size }. The scan_id is stored in sessionStorage("sastra_scan_id"). Both are read on scan-results.html.
- File type validation is done client-side by the accept attribute on the file input. The backend MUST also validate file type and size server-side (FR-5, FR-2).
- For large files or slow analysis, consider using a polling approach: POST /api/scans/run returns a scan_id immediately, then scan-results.html polls GET /api/scans/{scan_id}/status until "completed".
- ZIP files containing multiple source files should be extracted server-side and each file analysed individually. The results page shows per-file findings.
- The scanning overlay progress bar is a frontend animation only — it does not reflect real backend progress. For real progress, use WebSocket or Server-Sent Events.

7. Scan Results Page

scan-results.html

Displays the full detailed report for a completed scan. Reads scan_id from sessionStorage, fetches results from the backend, and renders a 3-card summary row plus an interactive findings table with clickable detail modals.

Page Sections

Page Header

Title "Scan Results", subtitle with filename and timestamp. "New Scan" outline button and "Export PDF" dark button.

Summary Card

Left card: 6 data fields — Total Findings, Risk Score (0-100, calculated from severity weights), Files Scanned, False Positives (ML-filtered), File, Language. Export PDF button at the bottom.

Severity Breakdown Card

Middle card: animated SVG donut chart split into Critical/High/Medium/Low segments with a colour legend showing counts on the right.

Top References Card

Right card: Two ranked bar chart groups — Top CWE (e.g. CWE-89, CWE-798) and Top OWASP (e.g. A03:2021, A07:2021). Bars animate in on page load.

Findings Table

Full-width table with columns: Severity (coloured pill + ML Confidence %), CWE ID, OWASP category, File, Line number, Issue title + AI fix preview, Code snippet. Filter chips above: All / Critical / High / Medium / Low.

Finding Detail Modal

Click any row to open a centred modal showing: severity pill, full title, file + line + CWE + OWASP + ML confidence metadata, full description paragraph, monospace code snippet block, full recommended fix text, and "AI-powered fix available" green tag.

Interactive Elements & Outputs

Element	Type	Action / Output
New Scan button (header)	Button	Navigates to new-scan.html.
Export PDF button (header)	Button	Calls exportPDF() — currently shows 2s loading animation. Wire to GET /api/scans/{id}/export
Export PDF button (summary card)	Button	Same as above — duplicate trigger.
All / Critical / High / Medium / Low chips	Filter buttons	Filters the findings table client-side. No API call — filtering is done on the pre-loaded allFindings
Finding row (click)	Clickable row	Opens openModal(finding) — renders the detail modal with full data from the allFindings array
Modal close (X button)	Button	Calls closeModal() — removes .show class from overlay.
View Full Report (modal)	Button	Placeholder. Wire to a dedicated report page or trigger PDF export.
Re-scan File (modal)	Button	Navigates to new-scan.html.

Required Backend API Calls

GET

/api/scans/{scan_id}/results

Fetches all data for the results page. Called on page load using `sastra_scan_id` from `sessionStorage`.

Response:

```
{ "total_findings": 12, "risk_score": 78, "files_scanned": 1, "false_positives": 0, "filename":  
"auth-service.py", "language": "Python", "counts": { "critical": 2, "high": 3, "medium": 4, "low": 3 },  
"top_cwe": [ {"label": "CWE-89", "count": 255}, ... ], "top_owasp": [ {"label": "A03:2021", "count": 150}, ... ],  
"findings": [ { "id": 1, "sev": "critical", "cwe": "CWE-89", "owasp": "A03:2021 - Injection", "file":  
"auth-service.py", "line": "84", "confidence": 100, "title": "SQL Injection", "desc": "...", "snippet":  
"...", "fix": "..." } ] }
```

GET

/api/scans/{scan_id}/export-pdf

Returns a generated PDF report as a binary stream for download.

Response:

```
Content-Type: application/pdf Content-Disposition: attachment; filename="sastra-report.pdf" [Binary PDF  
stream]
```

Integration Notes

- The `allFindings` array in `scan-results.html` is currently hardcoded with 12 sample findings. Replace this array with `data.findings` from the API response.
- Risk score is currently computed client-side: $(critical * 10 + high * 5 + medium * 2 + low * 1)$, capped at 99. Alternatively, compute server-side and return in the response.
- The ML Confidence % shown under each severity pill maps to the `finding.confidence` field. The ML classifier assigns this during analysis.
- The "AI-powered fix available" tag in the modal reflects that the fix was generated by the LLM API (FR-13). The fix text itself is the LLM output stored in the database.
- The pie chart and bar charts are built entirely from the findings data — no separate API calls needed for chart data if findings are included in the results response.

8. Scan History Page

scan-history.html

Shows a paginated, searchable, filterable list of all past scans for the authenticated user. Includes a summary strip, toolbar with search/filter/sort, a table of scans, and a slide-in detail drawer.

Page Sections

Navbar

Scan History link is active.

Page Header

"Scan History" title. "New Scan" button top-right.

Summary Strip

4 glassmorphism pills: Total Scans, Completed count, Total Vulnerabilities, Last Scan (relative time). Currently hardcoded — replace with API data.

Toolbar

Search input (filters by filename or language), filter buttons (All / Completed / Failed), sort dropdown (Newest / Oldest / Most Issues / Least Issues). All filtering is currently client-side on the static scans array.

History Table

Columns: File (icon + name + date + size), Status badge, Language pill, Severity tags, Issues count, Arrow button. Click arrow to open detail drawer.

Detail Drawer

Slides in from right on row arrow click. Shows: filename, date, Scan Details grid (status/language/size/total/critical/high), Vulnerabilities list (name + line per vuln), View Full Report button, Re-scan File button.

Pagination

Shows "X–Y of Z scans". Prev/Next and numbered page buttons. Currently paginates the client-side scans array.

Interactive Elements & Outputs

Element	Type	Action / Output
Search input	Input (text)	id="search-input". Calls filterScans() on input event. Filters by scan.name and scan.lang. For l
All / Completed / Failed chips	Filter buttons	Calls setFilter(). Adds ?status= param when wired to API.
Sort dropdown	Select	id="sort-select". Calls filterScans(). Values: newest, oldest, most-issues, least-issues. Add as
New Scan button	Button	Navigates to new-scan.html.
Row arrow button	Button	Calls openDrawer(scan_id). Populates detail drawer from the scans array.
View Full Report (drawer)	Button	Placeholder. Wire to scan-results.html?id={scan_id}.
Re-scan File (drawer)	Button	Navigates to new-scan.html.
Pagination buttons	Buttons	Increment/decrement currentPage, call filterScans(). For API pagination add ?page={n}&limit=

Required Backend API Calls

GET**/api/scans/history**

Returns paginated scan history for the authenticated user. Supports search, filter, and sort query params.

Response:

```
GET /api/scans/history?page=1&limit=8&status=completed&sort=newest&search=auth { "scans": [ { "id": "uuid", "name": "auth-service.py", "lang": "Python", "size": "45 KB", "status": "completed", "date": "2 days ago", "issues": 12, "critical": 2, "high": 4, "medium": 4, "low": 2, "vulns": [{ "sev": "critical", "name": "SQL Injection", "line": "Line 84" }] } ], "total": 24 }
```

GET**/api/scans/stats**

Returns aggregate stats for the 4 summary pills (same endpoint as dashboard).

Response:

```
{ "total_scans": 8, "completed_count": 7, "total_vulnerabilities": 73, "last_scan_ago": "2d" }
```

Integration Notes

- The scans array (8 entries) is hardcoded in the HTML. Replace with dynamic data from GET /api/scans/history.
- The detail drawer's vulns list reads from scan.vulns[] in the local array. If not included in the history response, add a separate endpoint: GET /api/scans/{id}/vulnerabilities.
- Server-side pagination is recommended for users with large scan histories. The current frontend pagination works on a client-side array.
- The "View Full Report" button in the drawer should navigate to scan-results.html?id={scan_id} — read the id query param on that page instead of sessionStorage.

9. Settings Page

settings.html

Allows authenticated users to update their username and password, and permanently delete their account. Settings link is active in the navbar.

Page Sections

Navbar

Settings link is active.

Page Header

"Settings" title + "Manage your profile and account preferences" subtitle.

Profile Settings Card

White glassmorphism card with a user icon header. Contains: single Username input (pre-filled from sessionStorage), New Password input, Confirm New Password input, Save Changes and Discard Changes buttons, and a green success toast.

Account Deletion Zone

Soft red warning zone below the card. Shows warning description. Delete Account button opens a confirmation modal.

Delete Confirmation Modal

Centred modal with warning icon, "Delete Account?" heading, explanation text, Cancel button, and "Yes, Delete" button.

Interactive Elements & Outputs

Element	Type	Action / Output
Username input	Input (text)	id="username". Pre-filled with sessionStorage.getItem("sastra_user"). Editable.
New Password input	Input (password)	id="new-password". Optional. Leave empty to keep current password.
Confirm New Password input	Input (password)	id="confirm-password". Must match new-password if filled.
Save Changes button	Button	Calls saveChanges(). Validates non-empty username and matching passwords. Calls PUT /api/user/update.
Discard Changes button	Button	Resets all inputs to their last saved values from sessionStorage. No API call.
Delete Account button	Button	Opens the confirmation modal (#delete-overlay). No API call yet.
Modal Cancel button	Button	Removes .show from the modal overlay. No API call.
Modal Yes, Delete button	Button	Currently redirects to signin.html. Wire to: DELETE /api/user/delete → sessionStorage.clear()

Required Backend API Calls

PUT	/api/user/update
-----	------------------

Updates the authenticated user's username and/or password. Password field is optional.

Request Body:

```
{ "username": "new_username", "password": "new_password" } (password is optional)
```

Response:

```
200: { "username": "new_username" } 400: { "error": "Username already taken" | "Password too short" }
```

DELETE**/api/user/delete**

Permanently deletes the user account and all associated scan data.

Response:

```
204: (no content) 401: { "error": "Unauthorized" }
```

Integration Notes

- The username input is pre-filled on page load from `sessionStorage("sastra_user")`. For extra accuracy, also call `GET /api/user/profile` on load to get the current server-side username.
- Password change is optional — if the `new-password` field is empty, omit the `password` field from the request body.
- After successful account deletion, clear both `sastra_user` and `sastra_token` from `sessionStorage` before redirecting.
- The success toast (`#save-toast`) auto-hides after 3 seconds via `setTimeout`. It shows "Changes saved successfully!".

10. Global Auth & sessionStorage Reference

All authenticated pages read user state from sessionStorage. The following keys must be set during login/signup and cleared on logout/delete.

Key	Set By	Read By	Value
sastra_user	signin.html, signup.html	All pages (navbar greeting)	Username string
sastra_token	signin.html, signup.html	All API calls	JWT or session token string
sastra_scan	new-scan.html (before redirect)	scan-results.html	JSON: { filename, lang, size }
sastra_scan_id	new-scan.html (on API success)	scan-results.html	UUID string of the completed scan

Auth Header Pattern

Include this header in all authenticated API calls:

```
Authorization: Bearer ${sessionStorage.getItem('sastra_token')}
```

Logout Pattern

Run this block in the Log Out button handler on every page:

```
await fetch('/api/auth/logout', { method: 'POST', headers: { 'Authorization': 'Bearer ' +
sessionStorage.getItem('sastra_token') } }); sessionStorage.clear(); window.location.href =
'signin.html';
```

11. Complete API Endpoint Summary

Method	Endpoint	Auth	Used On	Purpose
POST	/api/auth/login	No	signin.html	Authenticate user, return token
POST	/api/auth/register	No	signup.html	Create account, return token
POST	/api/auth/logout	Yes	All pages (nav)	Invalidate session token
GET	/api/scans/stats	Yes	dashboard.html, scan-history.html	Aggregate stats (totals, last scan)
GET	/api/scans/recent?limit=4	Yes	dashboard.html	Last 4 scans for dashboard table
POST	/api/scans/run	Yes	new-scan.html	Submit file/code for analysis
GET	/api/scans/{id}/results	Yes	scan-results.html	Full findings for a scan
GET	/api/scans/{id}/export-pdf	Yes	scan-results.html	Download PDF report
GET	/api/scans/history	Yes	scan-history.html	Paginated scan history
GET	/api/scans/{id}/vulnerabilities	Yes	scan-history.html (drawer)	Vulnerability list for a scan
PUT	/api/user/update	Yes	settings.html	Update username and/or password
DELETE	/api/user/delete	Yes	settings.html	Delete account permanently
GET	/api/user/profile	Yes	settings.html (optional)	Fetch current username

12. Database Schema Reference (SQLite)

Based on the functional requirements, the SQLite database should contain at minimum the following tables.

Table: users

id	INTEGER PRIMARY KEY AUTOINCREMENT
username	TEXT UNIQUE NOT NULL
password_hash	TEXT NOT NULL — bcrypt hash
created_at	DATETIME DEFAULT CURRENT_TIMESTAMP

Table: scans

id	TEXT PRIMARY KEY — UUID
user_id	INTEGER REFERENCES users(id)
filename	TEXT NOT NULL
language	TEXT — Python Java C++
file_size	TEXT
status	TEXT — completed failed
total_findings	INTEGER DEFAULT 0
risk_score	INTEGER DEFAULT 0
false_positives	INTEGER DEFAULT 0
scanned_at	DATETIME DEFAULT CURRENT_TIMESTAMP

Table: findings

id	INTEGER PRIMARY KEY AUTOINCREMENT
scan_id	TEXT REFERENCES scans(id)
severity	TEXT — critical high medium low
cwe_id	TEXT — e.g. CWE-89
owasp_category	TEXT — e.g. A03:2021
file_path	TEXT
line_number	INTEGER
title	TEXT
description	TEXT

<code>code_snippet</code>	TEXT
<code>suggested_fix</code>	TEXT — LLM-generated
<code>ml_confidence</code>	REAL — 0.0 to 1.0
<code>is_false_positive</code>	BOOLEAN DEFAULT 0 — ML filtered

Notes

- Passwords must never be stored in plain text. Use bcrypt with a work factor of at least 12.
- `scan_id` is a UUID (TEXT) so it can be safely exposed in URLs without revealing sequential IDs.
- The `suggested_fix` column stores the LLM API response for each finding. This avoids re-calling the LLM on every page load.
- `ml_confidence` stores the ML classifier's probability (0.0–1.0). The frontend displays it as a percentage (e.g. 0.85 → "ML 85%").
- `is_false_positive` is set by the ML false-positive filter. Findings with this flag are excluded from the `total_findings` count but stored for reference.